

Java Collection Framework — Important Methods by Class

Aim

To explore the Java Collection Framework by implementing and demonstrating the important methods of its core **classes** — `ArrayList`, `LinkedList`, `Vector`, `Stack`, `HashSet`, `LinkedHashSet`, `TreeSet`, `PriorityQueue`, `ArrayDeque`, `HashMap`, `LinkedHashMap`, `TreeMap`, and `Hashtable` — driven from one central runner class, `CollectionFrameworkRunner`.

Question:

 Java Collection Framework: Important Methods by **Class**

 `ArrayList` Class (Resizable-array implementation)

- `add(E e)` - Appends the specified element to the end of the list.
- `add(int index, E element)` - Inserts the specified element at the specified position.
- `get(int index)` - Returns the element at the specified position.
- `set(int index, E element)` - Replaces the element at the specified position with the new element.
- `remove(int index)` / `remove(Object o)` - Removes the element at the specified index or the first occurrence of the specified element.
- `contains(Object o)` - Returns true if this list contains the specified element.
- `size()` - Returns the number of elements in this list.
- `isEmpty()` - Returns true if this list contains no elements.
- `clear()` - Removes all elements from this list.
- `indexOf(Object o)` - Returns the index of the first occurrence of the specified element.
- `lastIndexOf(Object o)` - Returns the index of the last occurrence of the specified element.
- `sort(Comparator<? super E> c)` - Sorts the list according to the order induced by the specified Comparator.

 `LinkedList` Class (Doubly-linked list implementation)

- `add(E e)` - Appends the specified element to the end of this list.
- `addFirst(E e)` / `addLast(E e)` - Inserts the specified element at the beginning / end of this list.
- `get(int index)` - Returns the element at the specified position in this list.
- `getFirst()` / `getLast()` - Returns the first / last element in this list.

- `remove(int index)` / `remove(Object o)` - Removes the element at the specified position or the specified element.
- `removeFirst()` / `removeLast()` - Removes and returns the first / last element from this list.
- `offer(E e)` - Adds the specified element as the tail (last element) of this list.
- `poll()` - Retrieves and removes the head (first element) of this list, or returns null if empty.
- `peek()` - Retrieves, but does not remove, the head (first element) of this list, or returns null if empty.

📁 Vector Class (Synchronized, thread-safe resizable-array)

- `add(E e)` - Appends the specified element to the end of this Vector.
- `addElement(E obj)` - Adds the specified component to the end of this vector, increasing its size by one.
- `get(int index)` - Returns the element at the specified position.
- `set(int index, E element)` - Replaces the element at the specified position with the new element.
- `remove(int index)` - Removes the element at the specified position.
- `removeElement(Object obj)` - Removes the first (lowest-indexed) occurrence of the argument from this vector.
- `size()` - Returns the number of components in this vector.
- `capacity()` - Returns the current capacity of this vector.
- `contains(Object o)` - Returns true if this vector contains the specified element.

📁 Stack Class (Last-In-First-Out object stack)

- `push(E item)` - Pushes an item onto the top of this stack.
- `pop()` - Removes the object at the top of this stack and returns it.
- `peek()` - Looks at the object at the top of this stack without removing it.
- `empty()` - Tests if this stack is empty.
- `search(Object o)` - Returns the 1-based position where an object is on this stack.

📁 HashSet Class (Hash table based set, no duplicates, unordered)

- `add(E e)` - Adds the specified element to this set if it is not already present.
- `remove(Object o)` - Removes the specified element from this set if it is present.
- `contains(Object o)` - Returns true if this set contains the specified element.
- `size()` - Returns the number of elements in this set.
- `isEmpty()` - Returns true if this set contains no elements.
- `clear()` - Removes all elements from this set.

🔗 **LinkedHashSet Class** (Hash table and linked list implementation, maintains insertion order)

- `add(E e)` - Adds the specified element to this set if it is not already present.
- `remove(Object o)` - Removes the specified element from this set if it is present.
- `contains(Object o)` - Returns true if this set contains the specified element.
- `size()` - Returns the number of elements in this set.
- `clear()` - Removes all elements from this set.

🌲 **TreeSet Class** (NavigableSet implementation based on a TreeMap, sorted order)

- `add(E e)` - Adds the specified element to this set if it is not already present.
- `remove(Object o)` - Removes the specified element from this set if it is present.
- `contains(Object o)` - Returns true if this set contains the specified element.
- `first()` / `last()` - Returns the first (lowest) / last (highest) element currently in this set.
- `higher(E e)` / `lower(E e)` - Returns the least element strictly greater than / greatest element strictly less than the given element.
- `ceiling(E e)` / `floor(E e)` - Returns the least element greater than or equal to / greatest element less than or equal to the given element.
- `pollFirst()` / `pollLast()` - Retrieves and removes the first (lowest) / last (highest) element, or returns null if empty.

👑 **PriorityQueue Class** (An unbounded priority queue based on a priority heap)

- `add(E e)` / `offer(E e)` - Inserts the specified element into this priority queue.
- `peek()` - Retrieves, but does not remove, the head of this queue, or returns null if empty.
- `poll()` - Retrieves and removes the head of this queue, or returns null if empty.
- `remove(Object o)` - Removes a single instance of the specified element from this queue, if it is present.
- `contains(Object o)` - Returns true if this queue contains the specified element.
- `size()` - Returns the number of elements in this collection.

🔗 **ArrayDeque Class** (Resizable-array implementation of the Deque interface)

- `addFirst(E e) / addLast(E e)` - Inserts the specified element at the front / end of this deque.
- `offerFirst(E e) / offerLast(E e)` - Inserts the specified element at the front / end of this deque.
- `pollFirst() / pollLast()` - Retrieves and removes the first / last element of this deque, or returns null if empty.
- `peekFirst() / peekLast()` - Retrieves, but does not remove, the first / last element of this deque, or returns null if empty.

🔗 **HashMap Class** (Hash table based Map implementation, keys are unordered)

- `put(K key, V value)` - Associates the specified value with the specified key in this map.
- `get(Object key)` - Returns the value to which the specified key is mapped, or null if missing.
- `remove(Object key)` - Removes the mapping for the specified key from this map if present.
- `containsKey(Object key) / containsValue(Object value)` - Returns true if this map contains a mapping for the specified key / value.
- `keySet()` - Returns a Set view of the keys contained in this map.
- `values()` - Returns a Collection view of the values contained in this map.
- `entrySet()` - Returns a Set view of the key-value mappings contained in this map.
- `size()` - Returns the number of key-value mappings in this map.
- `isEmpty()` - Returns true if this map contains no key-value mappings.
- `clear()` - Removes all mappings from this map.
- `getOrDefault(Object key, V defaultValue)` - Returns the value mapped to the key, or defaultValue if the mapping doesn't exist.

🔗 **LinkedHashMap Class** (Hash table and linked list implementation of Map, maintains insertion order)

- `put(K key, V value)` - Associates the specified value with the specified key in this map.
- `get(Object key)` - Returns the value to which the specified key is mapped.
- `remove(Object key)` - Removes the mapping for the specified key from this map if present.
- `containsKey(Object key)` - Returns true if this map contains a mapping for the specified key.
- `keySet()` - Returns a Set view of the keys contained in this map.
- `values()` - Returns a Collection view of the values contained in this map.
- `entrySet()` - Returns a Set view of the key-value mappings contained in this map.

🌲🐘 TreeMap Class (Red-Black tree based NavigableMap implementation, keys are sorted)

- put(K key, V value) - Associates the specified value with the specified key in this map.
- get(Object key) - Returns the value to which the specified key is mapped.
- remove(Object key) - Removes the mapping for a key from this TreeMap if present.
- containsKey(Object key) / containsValue(Object value) - Returns true if this map contains a mapping for the specified key / value.
- firstKey() / lastKey() - Returns the first (lowest) / last (highest) key currently in this map.
- higherKey(K key) / lowerKey(K key) - Returns the least key strictly greater than / greatest key strictly less than the given key.
- ceilingKey(K key) / floorKey(K key) - Returns the least key greater than or equal to / greatest key less than or equal to the given key.
- entrySet() - Returns a Set view of the key-value mappings contained in this map.

🔒🐘 Hashtable Class (Synchronized, thread-safe key-value map implementation)

- put(K key, V value) - Maps the specified key to the specified value in this hashtable.
- get(Object key) - Returns the value to which the specified key is mapped.
- remove(Object key) - Removes the key (and its corresponding value) from this hashtable.
- containsKey(Object key) / containsValue(Object value) - Tests if the specified object is a key / value in this hashtable.
- keys() - Returns an enumeration of the keys in this hashtable.
- elements() - Returns an enumeration of the values in this hashtable.
- size() - Returns the number of keys in this hashtable.
- isEmpty() - Tests if this hashtable maps no keys to values.

🔧 System Requirements

Hardware

Standard PC / Laptop
Intel Core i3 / i5 processor or equivalent
Minimum 4 GB RAM
Minimum 500 MB free storage

Software

Operating System : Windows / Linux / macOS
Java Development Kit : JDK 17 or JDK 21
Text Editor or IDE : Eclipse, IntelliJ IDEA, NetBeans, or VS Code
Command Prompt / Terminal

1. CollectionFrameworkRunner.java — Driver Class

Source Code

```
package collectionspractice.classmethods;

public class CollectionFrameworkRunner {
    public static void main(String[] args) {
        System.out.println("Java Collection Framework - Important Methods by Class");

        ArrayListImplement.show();
        LinkedListImplement.show();
        VectorStackImplement.show();
        SetImplement.show();
        QueueDequeImplement.show();
        MapImplement.show();
        HashtableImplement.show();

        System.out.println("\nAll collection examples completed successfully.");
    }
}
```

This class is the **driver / runner** that ties all the demo classes together. Each collection class (ArrayListImplement, LinkedListImplement, etc.) has a **single** public static void show() method that prints a full demo of that collection. main() simply calls each show() method one after another, in a fixed order.

Exception in simple words: No exceptions are thrown or caught here. This class only performs **method calls in sequence**; any exception it could face would come from inside the show() methods it calls (none of which throw one under normal input, as explained in each section below).

2. Step-wise Execution

1. Program starts in CollectionFrameworkRunner.main().

```

2. Prints the title "Java Collection Framework - Important Methods by
Class".
3. Calls ArrayListImplement.show()    -> runs the full ArrayList demo.
4. Calls LinkedListImplement.show()    -> runs the full LinkedList demo.
5. Calls VectorStackImplement.show()   -> runs the Vector and Stack demo.
6. Calls SetImplement.show()           -> runs the
HashSet/LinkedHashSet/TreeSet demo.
7. Calls QueueDequeImplement.show()    -> runs the PriorityQueue/ArrayDeque
demo.
8. Calls MapImplement.show()           -> runs the
HashMap/LinkedHashMap/TreeMap demo.
9. Calls HashtableImplement.show()     -> runs the Hashtable demo.
10. Prints "All collection examples completed successfully."

```

3. Algorithm

```

START
  1. Print title
  2. CALL ArrayListImplement.show()
  3. CALL LinkedListImplement.show()
  4. CALL VectorStackImplement.show()
  5. CALL SetImplement.show()
  6. CALL QueueDequeImplement.show()
  7. CALL MapImplement.show()
  8. CALL HashtableImplement.show()
  9. Print "All collection examples completed successfully."
STOP

```

4. UML Class Diagram



Output:

```
PS C:\Users\Admin\OneDrive\Documents\basicjava> & 'C:\Program Files\Java\jdk-26.0.1\bin\java.exe' '--enable-preview' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\Admin\OneDrive\Documents\basicjava\bin' 'collectionspractice.classmethods.collectionframeworkrunner'
Java Collection Framework - Important Methods by Class

===*** ArrayList Demo ***===
List after adding elements: [Elon musk, virat, Samurai, Elon musk]
Student at index 0: Elon musk
After replacing index 0: [Meera, virat, Samurai, Elon musk]
Contains virat? true
First index of Asha: 3
Last index of Asha: 3
After removing Elon musk: [Meera, virat, Samurai]
List size: 3
Is list empty? false
Sorted list: [Meera, Samurai, virat]
After clear(): []
Now empty? true

===*** LinkedList Demo ***===
Daily tasks: [Drink water, Wake up, Study Java, Sleep]
First task: Drink water
Last task: Sleep
After offer(): [Drink water, Wake up, Study Java, Sleep, Review notes]
Peeked head: Drink water
Removed first task: Drink water
Tasks after poll(): [Wake up, Study Java, Sleep, Review notes]
Removed first element: Wake up
Removed last element: Review notes
Final task list: [Study Java, Sleep]

=== Vector and Stack Demo ===
Vector values: [Aman, Babji, Charlie]
Name at index 1: Babji
After set(): [Aman, Bharat, Charlie]
After remove(index): [Aman, Bharat]
Contains Alice? true
Vector size: 2
Vector capacity: 10
Books stack: [Java Basics, OOP, Collections]
Top element without removing: Collections
Removed top element: Collections
```

```
Removed last element: Review notes
Final task list: [Study Java, Sleep]

=== Vector and Stack Demo ===
Vector values: [Aman, Babji, Charlie]
Name at index 1: Babji
After set(): [Aman, Bharat, Charlie]
After remove(index): [Aman, Bharat]
Contains Alice? true
Vector size: 2
Vector capacity: 10
Books stack: [Java Basics, OOP, Collections]
Top element without removing: Collections
Removed top element: Collections
Stack after pop: [Java Basics, OOP]
Is stack empty? false
Search for OOP: 1

=== Set Demo ===
HashSet fruits: [Apple, Mango, Banana]
Contains Apple? true
Set size: 3
After removing Banana: [Apple, Mango]
LinkedHashSet order: [Delhi, Mumbai, Chennai]
TreeSet numbers: [10, 20, 30]
First element: 10
Last element: 30
After clear(): []

===*** Queue and Deque ***===
PriorityQueue: [10, 40, 25]
Head element: 10
Removed head: 10
Queue after poll: [25, 40]
Deque content: [C++, Java, Python]
First element: C++
Last element: Python
Removed from front: C++
Removed from back: Python
Final deque: [Java]
```

```

===Map Demo===
Student map: {101-Prabhash, 102-Ravi, 103-Prabhash}
Value for key 101: Prabhash
Contains key 102? true
Contains value Ravi? true
Keys: [101, 102, 103]
Values: [Prabhash, Ravi, Prabhash]
Entries: [101-Prabhash, 102-Ravi, 103-Prabhash]
After removing 103: {101-Prabhash, 102-Ravi}
LinkedHashMap order: {Math=90, Science=88, English=95}
TreeMap sorted keys: {Food=5000, Rent=10000, Travel=2000}
First key: Food
Last key: Travel

===Hashtable Demo===
Stock table: {Tablet=5, Mobile=10, Laptop=2}
Available laptops: 2
Contains key Mobile? true
Contains value 5? true
After removing Tablet: {Mobile=10, Laptop=2}
Total items: 2
Is map empty? false
Keys: [Mobile, Laptop]
Values: [10, 2]

```

```

All collection examples completed successfully.
PS C:\Users\Admin\OneDrive\Documents\basicjava>

```

2. ArrayListImplement.java — ArrayList

Source Code

```

package collectionspractice.classmethods;

import java.util.ArrayList;
import java.util.Comparator;
import java.util.List;

public class ArrayListImplement {

    public static void show() {
        System.out.println("\n===*** ArrayList Demo ***===");

        // ArrayList is a dynamic array. It grows or shrinks automatically
        when needed.
        List<String> students = new ArrayList<>();

        // add element to the end
        students.add("Elon musk");
        students.add("Samurai");

        // insert at a specific position
        students.add(1, "virat");
        students.add("Elon musk");

        System.out.println("List after adding elements: " + students);

        // read element by position
        System.out.println("Student at index 0: " + students.get(0));

        // replace existing value
    }
}

```

```

students.set(0, "Meera");
System.out.println("After replacing index 0: " + students);

// search for an element
System.out.println("Contains virat? " + students.contains("virat"));

// indexOf and lastIndexOf helps to find duplicate positions
System.out.println("First index of Asha: " + students.indexOf("Elon
musk"));
System.out.println("Last index of Asha: " +
students.lastIndexOf("Elon musk"));

// delete an element
students.remove("Elon musk");
System.out.println("After removing Elon musk: " + students);

// total number of elements
System.out.println("List size: " + students.size());

// checks if list has no elements
System.out.println("Is list empty? " + students.isEmpty());

// arrange elements in order
students.sort(Comparator.naturalOrder());
System.out.println("Sorted list: " + students);

// removes everything from the list
students.clear();
System.out.println("After clear(): " + students);
System.out.println("Now empty? " + students.isEmpty());
}
}

```

`ArrayList` is a **dynamic (resizable) array** — it implements `List`, keeps insertion order, allows duplicates, and grows automatically as elements are added.

Method	What it does
<code>add(e) / add(i, e)</code>	Appends, or inserts at a specific index
<code>get(i)</code>	Reads the element at index <code>i</code>
<code>set(i, e)</code>	Replaces the element at index <code>i</code>
<code>contains()</code>	Checks if a value exists
<code>indexOf() / lastIndexOf()</code>	First / last position of a value
<code>remove(Object)</code>	Removes the first matching value
<code>size() / isEmpty()</code>	Element count / emptiness check
<code>sort(Comparator)</code>	Sorts the list in place
<code>clear()</code>	Removes all elements

Exception in simple words: No exception is thrown or caught in this program. `get(i)` and `add(i, e)` would throw `IndexOutOfBoundsException` for an out-of-range index — every index used here (0, 1) is valid when it's called.

Note: the print labels say "First index of Asha" / "Last index of Asha", but the code actually calls `indexOf("Elon musk") / lastIndexOf("Elon musk")` — the label text does not match what is searched for. This is a **labeling/comment mismatch**, not a compiler error, so the program still runs correctly and prints the index of "Elon musk".

2. Step-wise Execution

```
1. ArrayList<String> students created (via List reference).
2. add("Elon musk") -> [Elon musk]
3. add("Samurai") -> [Elon musk, Samurai]
4. add(1,"virat") -> [Elon musk, virat, Samurai]
5. add("Elon musk") -> [Elon musk, virat, Samurai, Elon musk]
6. Print list: [Elon musk, virat, Samurai, Elon musk]
7. get(0) = "Elon musk"
8. set(0,"Meera") -> [Meera, virat, Samurai, Elon musk]
9. contains("virat") = true
10. indexOf("Elon musk") = 3 (first occurrence, after Meera replaced index 0)
11. lastIndexOf("Elon musk") = 3 (only one occurrence remains, so same index)
12. remove("Elon musk") removes that element -> [Meera, virat, Samurai]
13. size() = 3
14. isEmpty() = false
15. sort(naturalOrder) -> alphabetical: [Meera, Samurai, virat]
    (uppercase letters sort before lowercase in natural String order,
    so "Meera" and "Samurai" come before "virat")
16. Print sorted list: [Meera, Samurai, virat]
17. clear() -> []
18. isEmpty() = true
```

3. Algorithm

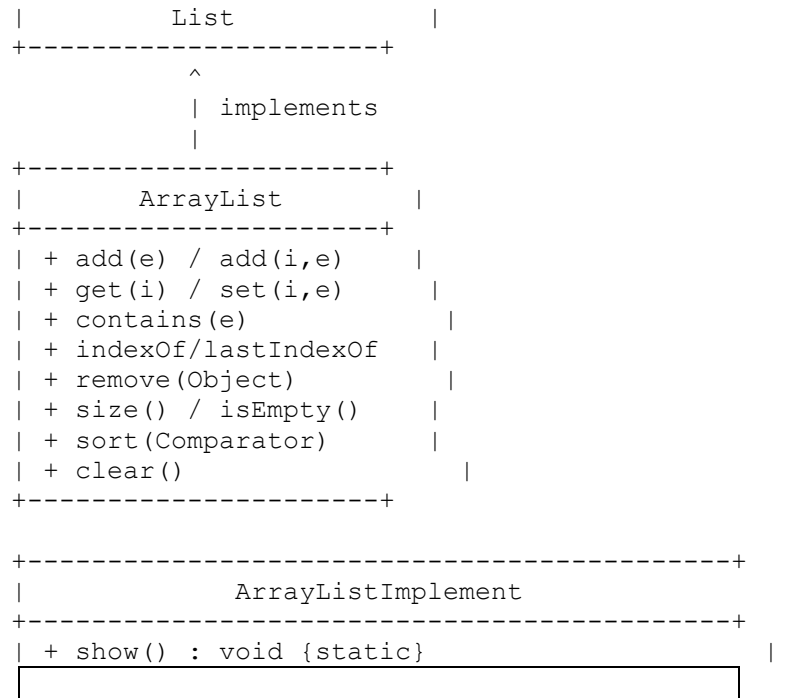
START

1. Create ArrayList students
2. add, add(index,e) to build the list
3. Print list; print get(0)
4. set(0, "Meera"); print list
5. Print contains("virat")
6. Print indexOf("Elon musk"), lastIndexOf("Elon musk")
7. remove("Elon musk"); print list
8. Print size(), isEmpty()
9. sort(naturalOrder); print sorted list
10. clear(); print list and isEmpty()

STOP

4. UML Class Diagram

```
+-----+
|    <<interface>>    |
```



3. LinkedListImplement.java — LinkedList

Source Code

```

package collectionspractice.classmethods;

import java.util.LinkedList;

public class LinkedListImplement {

    public static void show() {
        System.out.println("\n===@@@ LinkedList Demo @@@===");

        // LinkedList stores elements in nodes
        LinkedList<String> tasks = new LinkedList<>();

        // insert at end
        tasks.add("Wake up");
        tasks.add("Study Java");

        // insert at the front or back
        tasks.addFirst("Drink water");
    }
}

```

```

tasks.addLast("Sleep");

System.out.println("Daily tasks: " + tasks);

// returns first/last elements
System.out.println("First task: " + tasks.getFirst());
System.out.println("Last task: " + tasks.getLast());

// add element at the end like queue behavior
tasks.offer("Review notes");
System.out.println("After offer(): " + tasks);

// view head
System.out.println("Peeked head: " + tasks.peek());

// remove and return first element
String firstTask = tasks.poll();
System.out.println("Removed first task: " + firstTask);
System.out.println("Tasks after poll(): " + tasks);

System.out.println("Removed first element: " + tasks.removeFirst());
System.out.println("Removed last element: " + tasks.removeLast());
System.out.println("Final task list: " + tasks);
}
}

```

`LinkedList` stores elements as **linked nodes** rather than an array, and implements both `List` and `Deque`. It supports insertion/removal at both ends efficiently, along with Queue-style methods (`offer`, `peek`, `poll`).

Method	What it does
<code>add(e)</code>	Appends to the end
<code>addFirst(e) / addLast(e)</code>	Inserts at front / back
<code>getFirst() / getLast()</code>	Returns first/last, throws <code>NoSuchElementException</code> if empty
<code>offer(e)</code>	Inserts at the end (Queue-style)
<code>peek()</code>	Looks at the head, returns <code>null</code> if empty
<code>poll()</code>	Removes & returns the head, returns <code>null</code> if empty
<code>removeFirst() / removeLast()</code>	Removes & returns front/back, throws <code>NoSuchElementException</code> if empty

Exception in simple words: No exception occurs in this run because `getFirst()`, `getLast()`, `removeFirst()`, and `removeLast()` are only called while the list still has elements. If any of them were called on an **empty** list, Java would throw `NoSuchElementException` (unchecked runtime exception).

2. Step-wise Execution

1. `LinkedList<String> tasks` created.
2. `add("Wake up")` → `[Wake up]`

```

3. add("Study Java")      -> [Wake up, Study Java]
4. addFirst("Drink water") -> [Drink water, Wake up, Study Java]
5. addLast("Sleep")       -> [Drink water, Wake up, Study Java, Sleep]
6. Print tasks: [Drink water, Wake up, Study Java, Sleep]
7. getFirst() = "Drink water"
8. getLast() = "Sleep"
9. offer("Review notes") -> [Drink water, Wake up, Study Java, Sleep, Review
notes]
10. Print after offer(): [Drink water, Wake up, Study Java, Sleep, Review
notes]
11. peek() = "Drink water" (head, not removed)
12. poll() removes & returns "Drink water" -> [Wake up, Study Java, Sleep,
Review notes]
13. Print removed first task: "Drink water"
14. Print tasks after poll(): [Wake up, Study Java, Sleep, Review notes]
15. removeFirst() removes & returns "Wake up" -> [Study Java, Sleep, Review
notes]
16. removeLast() removes & returns "Review notes" -> [Study Java, Sleep]
17. Print final task list: [Study Java, Sleep]

```

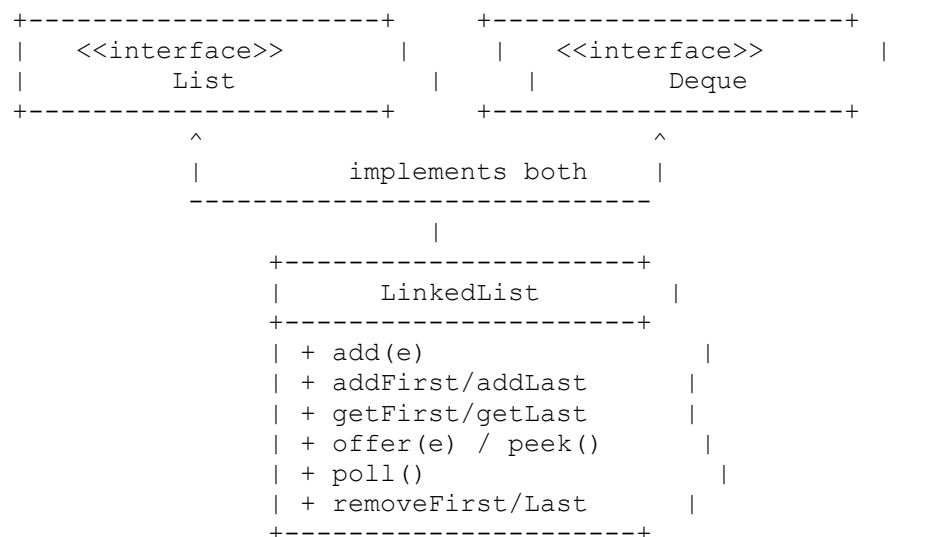
3. Algorithm

START

1. Create LinkedList tasks
2. add, addFirst, addLast to build the list
3. Print tasks; print getFirst(), getLast()
4. offer("Review notes"); print tasks
5. Print peek()
6. poll() -> store and print removed element; print tasks
7. removeFirst() -> print removed element
8. removeLast() -> print removed element
9. Print final tasks

STOP

4. UML Class Diagram



+-----+			
LinkedListImplement			
+-----+			
+ show() : void {static}			
<table><tr><td> </td></tr><tr><td> </td></tr></table>			

4. VectorStackImplement.java — Vector and Stack

Source Code

```
package collectionspractice.classmethods;

import java.util.Stack;
import java.util.Vector;

public class VectorStackImplement {

    public static void show() {
        System.out.println("\n== Vector and Stack Demo ==");

        // Vector is a synchronized list and works like ArrayList.
        Vector<String> names = new Vector<>();
        names.add("Aman");
        names.add("Babji");
        names.add("Charlie");

        System.out.println("Vector values: " + names);
        System.out.println("Name at index 1: " + names.get(1));
        names.set(1, "Bharat");
        System.out.println("After set(): " + names);

        // remove by position
        names.remove(2);
        System.out.println("After remove(index): " + names);

        // check if a value exists
        System.out.println("Contains Alice? " + names.contains("Aman"));
        System.out.println("Vector size: " + names.size());
        System.out.println("Vector capacity: " + names.capacity());

        // Stack works on LIFO principle (Last In First Out)
```

```

Stack<String> books = new Stack<>();
books.push("Java Basics");
books.push("OOP");
books.push("Collections");

System.out.println("Books stack: " + books);
System.out.println("Top element without removing: " + books.peek());
System.out.println("Removed top element: " + books.pop());
System.out.println("Stack after pop: " + books);
System.out.println("Is stack empty? " + books.empty());
System.out.println("Search for OOP: " + books.search("OOP"));
    }
}

```

Vector and Stack are both **legacy, synchronized (thread-safe) classes**. Vector behaves like a thread-safe ArrayList. Stack **extends vector** and adds LIFO (Last In, First Out) behaviour — think of a pile of books where you take from the top.

Method	Class	What it does
add(e) / get(i) / set(i,e)	Vector	Standard List-style access
remove(int index)	Vector	Removes element at that index
contains() / size()	Vector	Checks presence / counts elements
capacity()	Vector	Internal array size (may exceed element count)
push(e)	Stack	Adds to the top
peek()	Stack	Looks at the top, throws EmptyStackException if empty
pop()	Stack	Removes & returns the top, throws EmptyStackException if empty
empty()	Stack	Checks if the stack has no elements
search(e)	Stack	1-based position from top, or -1 if not found

Exception: No exception occurs in this run — peek() and pop() are only called while the stack still has books. Calling peek()/pop() on an **empty** stack would throw EmptyStackException (unchecked runtime exception). Also note the comment says "check if a value exists" but prints "Contains Alice?" while actually testing contains("Aman") — another label/code mismatch, not an error.

2. Step-wise Execution

VECTOR

1. Vector<String> names created.
2. add("Aman"), add("Babji"), add("Charlie")
-> [Aman, Babji, Charlie]
3. Print vector: [Aman, Babji, Charlie]
4. get(1) = "Babji"
5. set(1,"Bharat") -> [Aman, Bharat, Charlie]
6. remove(2) removes index 2 ("Charlie") -> [Aman, Bharat]

```

7.  contains("Aman") = true
8.  size() = 2
9.  capacity() prints internal array size (default starts at 10,
    so likely 10 unless it needed to grow)

```

STACK

```

10. Stack<String> books created.
11. push("Java Basics") -> [Java Basics]
12. push("OOP")           -> [Java Basics, OOP]
13. push("Collections")  -> [Java Basics, OOP, Collections]
14. Print stack: [Java Basics, OOP, Collections]
15. peek() = "Collections" (top, not removed)
16. pop() removes & returns "Collections" -> [Java Basics, OOP]
17. Print stack after pop: [Java Basics, OOP]
18. empty() = false
19. search("OOP") = 2 (2nd from the top: OOP=1... wait, top is now
    "OOP" after pop, so OOP is at position 1 from the top)

```

3. Algorithm

START

PART A – Vector

```

1. Create Vector names
2. add three names
3. Print vector; print get(1)
4. set(1, "Bharat"); print vector
5. remove(2); print vector
6. Print contains("Aman"), size(), capacity()

```

PART B – Stack

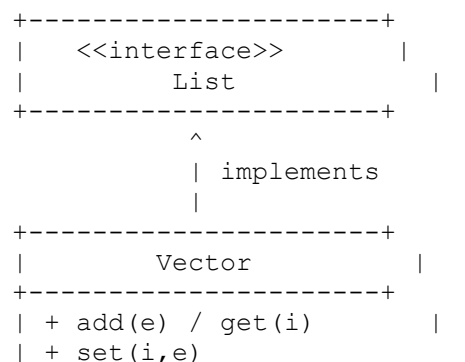
```

7. Create Stack books
8. push three books
9. Print stack
10. peek() -> print top element
11. pop() -> print & remove top element
12. Print stack after pop
13. Print empty()
14. search("OOP") -> print position from top

```

STOP

4. UML Class Diagram



```

| + remove(int)          |
| + contains(e) / size() |
| + capacity() : int     |
+-----+
      ^
      | extends
      |
+-----+
|          Stack          |
+-----+
| + push(e) : E           |
| + pop() : E             |
|   throws EmptyStack-   |
|   Exception            |
| + peek() : E           |
|   throws EmptyStack-   |
|   Exception            |
| + empty() : boolean     |
| + search(e) : int       |
+-----+

+-----+
|          VectorStackImplement      |
+-----+
| + show() : void {static}           |
+-----+

```

5. SetImplement.java — HashSet, LinkedHashSet, TreeSet

Source Code

```

package collectionspractice.classmethods;

import java.util.HashSet;
import java.util.LinkedHashSet;
import java.util.Set;
import java.util.TreeSet;

public class SetImplement {

    public static void show() {
        System.out.println("\n=== Set Demo ===");
    }
}

```

```

// HashSet: no duplicates, no order guarantee
Set<String> fruits = new HashSet<>();
fruits.add("Apple");
fruits.add("Mango");
fruits.add("Apple");
fruits.add("Banana");

System.out.println("HashSet fruits: " + fruits);
System.out.println("Contains Apple? " + fruits.contains("Apple"));
System.out.println("Set size: " + fruits.size());
fruits.remove("Banana");
System.out.println("After removing Banana: " + fruits);

// LinkedHashSet keeps insertion order
Set<String> cities = new LinkedHashSet<>();
cities.add("Delhi");
cities.add("Mumbai");
cities.add("Chennai");
System.out.println("LinkedHashSet order: " + cities);

// TreeSet: stores in sorted order
Set<Integer> numbers = new TreeSet<>();
numbers.add(30);
numbers.add(10);
numbers.add(20);
numbers.add(10);

System.out.println("TreeSet numbers: " + numbers);
System.out.println("First element: " + ((TreeSet<Integer>)
numbers).first());
System.out.println("Last element: " + ((TreeSet<Integer>)
numbers).last());

numbers.clear();
System.out.println("After clear(): " + numbers);
}
}

```

This program compares **three flavours of set**, all of which reject duplicates but differ in ordering:

Class	Order kept
HashSet	No guaranteed order (based on hash codes)
LinkedHashSet	Insertion order (the order elements were added)
TreeSet	Sorted order (ascending, using natural ordering or a Comparator)

The `numbers` variable is declared as `Set<Integer>` but the object is actually a `TreeSet`, so it is **cast back to `TreeSet<Integer>`** — `((TreeSet<Integer>) numbers)` — to call `first()/last()`, which exist on `TreeSet/NavigableSet` but not on the plain `Set` interface.

Exception in simple words: No exception occurs here. This **downcast** (`TreeSet<Integer>`) `numbers` is safe because the object really is a `TreeSet` at runtime; if `numbers` had actually held a different `Set` implementation (like `HashSet`), this same cast would throw `ClassCastException` (unchecked runtime exception) — "you tried to treat an object as a type it isn't."

2. Step-wise Execution

HASHSET

1. `HashSet<String> fruits` created.
2. `add("Apple"), add("Mango"), add("Apple" - ignored, duplicate), add("Banana")`
-> {Apple, Mango, Banana} (order not guaranteed when printed)
3. `contains("Apple") = true`
4. `size() = 3`
5. `remove("Banana")` -> {Apple, Mango}

LINKEDHASHSET

6. `LinkedHashSet<String> cities` created.
7. `add("Delhi"), add("Mumbai"), add("Chennai")`
-> prints EXACTLY in insertion order: [Delhi, Mumbai, Chennai]

TREESET

8. `TreeSet<Integer> numbers` created (declared as `Set<Integer>`).
9. `add(30), add(10), add(20), add(10 - ignored, duplicate)`
-> internally sorted: {10, 20, 30}
10. Print numbers: [10, 20, 30]
11. Cast to `TreeSet`, call `first() = 10`
12. Cast to `TreeSet`, call `last() = 30`
13. `clear()` -> {}
14. Print after `clear()`: []

3. Algorithm

START

PART A – HashSet

1. Create `HashSet` `fruits`
2. add four values (one duplicate ignored)
3. Print set, `contains()`, `size()`
4. `remove("Banana");` print set

PART B – LinkedHashSet

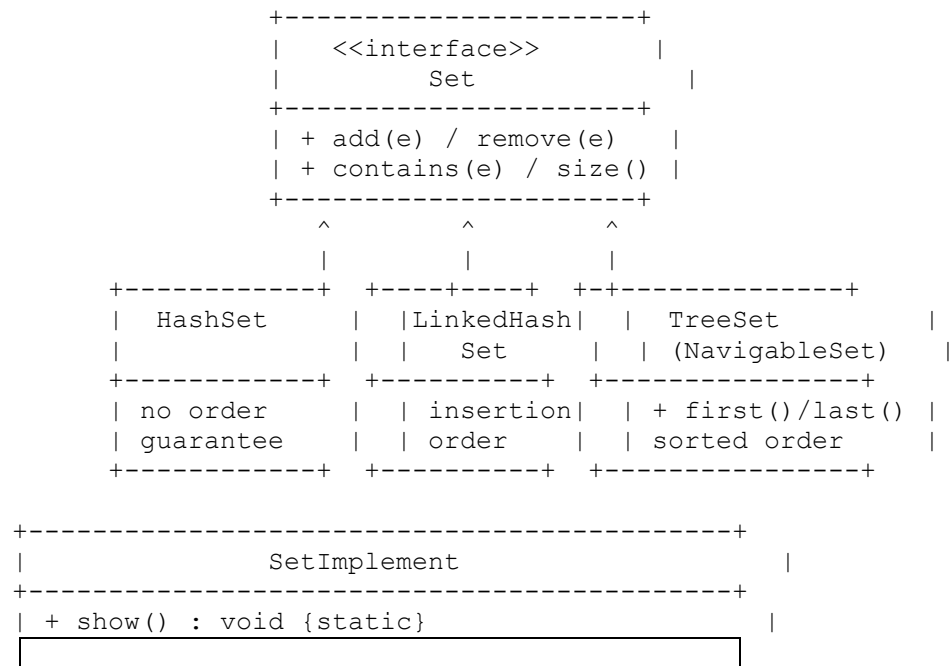
5. Create `LinkedHashSet` `cities`
6. add three cities
7. Print set (in insertion order)

PART C – TreeSet

8. Create `TreeSet` `numbers` (Set reference)
9. add four values (one duplicate ignored, auto-sorted)
10. Print set
11. Cast to `TreeSet`; print `first()`, `last()`
12. `clear();` print set

STOP

4. UML Class Diagram



6. QueueDequeImplement.java — PriorityQueue and ArrayDeque

Source Code

```
package collectionspractice.classmethods;

import java.util.ArrayDeque;
import java.util.PriorityQueue;
import java.util.Queue;

public class QueueDequeImplement {

    public static void show() {
        System.out.println("\n===$$$ Queue and Deque $$$===");

        // PriorityQueue gives elements in order according to priority.
        Queue<Integer> priorities = new PriorityQueue<>();
        priorities.offer(40);
    }
}
```

```

priorities.offer(10);
priorities.offer(25);

System.out.println("PriorityQueue: " + priorities);
System.out.println("Head element: " + priorities.peek());
System.out.println("Removed head: " + priorities.poll());
System.out.println("Queue after poll: " + priorities);

// ArrayDeque works as a double-ended queue.
ArrayDeque<String> deck = new ArrayDeque<>();
deck.addFirst("Java");
deck.addLast("Python");
deck.offerFirst("C++");

System.out.println("Deque content: " + deck);
System.out.println("First element: " + deck.peekFirst());
System.out.println("Last element: " + deck.peekLast());
System.out.println("Removed from front: " + deck.pollFirst());
System.out.println("Removed from back: " + deck.pollLast());
System.out.println("Final deque: " + deck);
}
}

```

`PriorityQueue` is a **Queue** that does not store elements in insertion order — instead, the **smallest value (by natural ordering) always comes out first** when you `poll()` or `peek()`. Internally it uses a data structure called a **heap**, so printing it directly does **not** show a fully sorted list — only the *head* (index 0 of its internal array) is guaranteed to be the smallest.

`ArrayDeque` (used here as a `Deque`) allows adding/removing from **both ends** and does print elements in actual front-to-back order.

Method	Class	What it does
<code>offer(e)</code>	<code>PriorityQueue</code>	Inserts, keeping heap order internally
<code>peek()</code>	<code>PriorityQueue</code>	Returns the smallest element (head), without removing
<code>poll()</code>	<code>PriorityQueue</code>	Removes & returns the smallest element
<code>addFirst(e) / addLast(e)</code>	<code>ArrayDeque</code>	Inserts at front / back
<code>offerFirst(e)</code>	<code>ArrayDeque</code>	Inserts at front (Deque-style)
<code>peekFirst() / peekLast()</code>	<code>ArrayDeque</code>	Look at front/back without removing
<code>pollFirst() / pollLast()</code>	<code>ArrayDeque</code>	Removes & returns front/back

Exception: No exception occurs in this run. `peek()/poll()` on `PriorityQueue`, and `peekFirst/peekLast/pollFirst/pollLast` on `ArrayDeque`, all return `null` (not an exception) when the structure is empty — this program never calls them on an empty structure anyway.

2. Step-wise Execution

PRIORITYQUEUE

1. PriorityQueue<Integer> priorities created.
2. offer(40) -> heap: [40]
3. offer(10) -> heap re-arranges so smallest is at the head: [10, 40]
4. offer(25) -> heap: [10, 40, 25] (internal array order, NOT fully sorted – only guarantee is index 0 is the smallest)
5. Print priorities: [10, 40, 25] (heap array order, may vary by JVM but head is always the minimum)
6. peek() = 10 (the smallest, always at the head)
7. poll() removes & returns 10 -> heap re-arranges: [25, 40]
8. Print queue after poll: [25, 40]

ARRAYDEQUE

9. ArrayDeque<String> deck created.
10. addFirst("Java") -> [Java]
11. addLast("Python") -> [Java, Python]
12. offerFirst("C++") -> [C++, Java, Python]
13. Print deck: [C++, Java, Python]
14. peekFirst() = "C++"
15. peekLast() = "Python"
16. pollFirst() removes & returns "C++" -> [Java, Python]
17. pollLast() removes & returns "Python" -> [Java]
18. Print final deque: [Java]

3. Algorithm

START

PART A – PriorityQueue

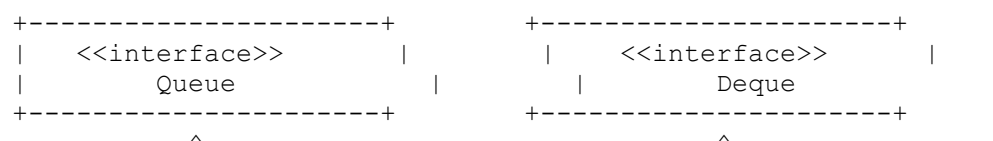
1. Create PriorityQueue priorities
2. offer(40), offer(10), offer(25)
3. Print queue (heap-array order)
4. Print peek() -> smallest element
5. poll() -> remove & print smallest element
6. Print queue after poll

PART B – ArrayDeque

7. Create ArrayDeque deck
8. addFirst, addLast, offerFirst to build the deque
9. Print deck; print peekFirst(), peekLast()
10. pollFirst() -> print & remove front
11. pollLast() -> print & remove back
12. Print final deque

STOP

4. UML Class Diagram



	implements		implements
+-----+		+-----+	
PriorityQueue		ArrayDeque	
+-----+		+-----+	
+ offer(e)		+ addFirst/addLast	
+ peek() : E (smallest)		+ offerFirst(e)	
+ poll() : E (smallest)		+ peekFirst/peekLast	
(heap-ordered)		+ pollFirst/pollLast	
+-----+		+-----+	
+-----+			
QueueDequeImplement			
+-----+			
+ show() : void {static}			

7. MapImplement.java — HashMap, LinkedHashMap, TreeMap

Source Code

```
package collectionspractice.classmethods;

import java.util.HashMap;
import java.util.LinkedHashMap;
import java.util.Map;
import java.util.TreeMap;

public class MapImplement {

    public static void show() {
        System.out.println("\n===&&& Map Demo &&&===");

        // HashMap stores key-value pairs without maintaining the order
        Map<Integer, String> students = new HashMap<>();
        students.put(101, "Prabhash");
        students.put(102, "Ravi");
        students.put(103, "Prabhash");

        System.out.println("Student map: " + students);
        System.out.println("Value for key 101: " + students.get(101));
        System.out.println("Contains key 102? " + students.containsKey(102));
    }
}
```

```

        System.out.println("Contains value Ravi? " +
students.containsValue("Ravi"));
        System.out.println("Keys: " + students.keySet());
        System.out.println("Values: " + students.values());
        System.out.println("Entries: " + students.entrySet());

        students.remove(103);
        System.out.println("After removing 103: " + students);

        // LinkedHashMap keeps insertion order.
        Map<String, Integer> marks = new LinkedHashMap<>();
        marks.put("Math", 90);
        marks.put("Science", 88);
        marks.put("English", 95);
        System.out.println("LinkedHashMap order: " + marks);

        // TreeMap stores keys in sorted order.
        Map<String, Integer> budget = new TreeMap<>();
        budget.put("Food", 5000);
        budget.put("Travel", 2000);
        budget.put("Rent", 10000);
        System.out.println("TreeMap sorted keys: " + budget);
        System.out.println("First key: " + ((TreeMap<String, Integer>)
budget).firstKey());
        System.out.println("Last key: " + ((TreeMap<String, Integer>)
budget).lastKey());
    }
}

```

This program compares **three flavours of Map**, all storing key-value pairs with unique keys, but differing in ordering — same idea as the Set comparison above:

Class	Order kept
HashMap	No guaranteed order
LinkedHashMap	Insertion order
TreeMap	Sorted order by key

Note: keys 101, 102, 103 are all **different keys** even though 101 and 103 both map to the **same value**, "Prabhash" — a Map only forbids duplicate *keys*, not duplicate *values*.

The `budget` variable is declared as `Map<String, Integer>` but the object is actually a `TreeMap`, so it is cast back — `((TreeMap<String, Integer>) budget)` — to call `firstKey()/lastKey()`, which exist on `TreeMap/NavigableMap` but not the plain `Map` interface.

Exception: No exception occurs here. As with the `TreeSet` cast earlier, this downcast is safe because the object really is a `TreeMap`; casting to the wrong type would throw `ClassCastException` (unchecked runtime exception).

2. Step-wise Execution

HASHMAP

1. `HashMap<Integer,String> students` created.
2. `put(101,"Prabhash"), put(102,"Ravi"), put(103,"Prabhash")`
-> {101=Prabhash, 102=Ravi, 103=Prabhash} (print order not guaranteed)
3. `get(101) = "Prabhash"`
4. `containsKey(102) = true`
5. `containsValue("Ravi") = true`
6. `keySet() = [101,102,103]` (order not guaranteed)
7. `values() = [Prabhash,Ravi,Prabhash]`
8. `entrySet() = [101=Prabhash, 102=Ravi, 103=Prabhash]`
9. `remove(103) -> {101=Prabhash, 102=Ravi}`

LINKEDHASHMAP

10. `LinkedHashMap<String,Integer> marks` created.
11. `put("Math",90), put("Science",88), put("English",95)`
-> prints EXACTLY in insertion order: {Math=90, Science=88, English=95}

TREEMAP

12. `TreeMap<String,Integer> budget` created (declared as `Map`).
13. `put("Food",5000), put("Travel",2000), put("Rent",10000)`
-> sorted alphabetically by key: {Food=5000, Rent=10000, Travel=2000}
14. Print budget: {Food=5000, Rent=10000, Travel=2000}
15. Cast to `TreeMap`, `firstKey() = "Food"`
16. Cast to `TreeMap`, `lastKey() = "Travel"`

3. Algorithm

START

PART A – HashMap

1. Create `HashMap` `students`
2. put three roll-number/name pairs
3. Print `map`, `get()`, `containsKey()`, `containsValue()`
4. Print `keySet()`, `values()`, `entrySet()`
5. `remove(103)`; print `map`

PART B – LinkedHashMap

6. Create `LinkedHashMap` `marks`
7. put three subject/mark pairs
8. Print `map` (in insertion order)

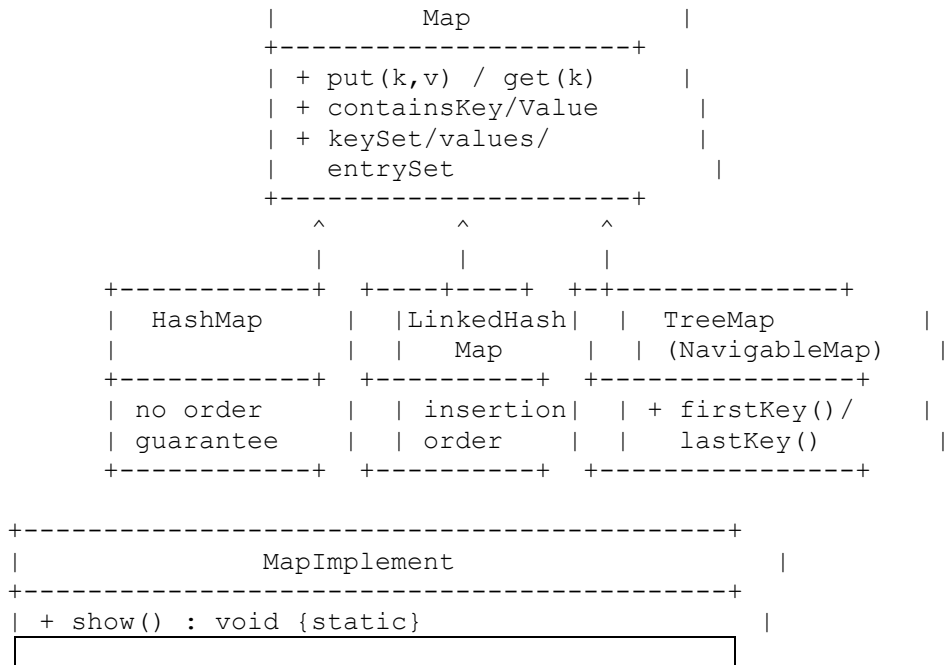
PART C – TreeMap

9. Create `TreeMap` `budget` (`Map` reference)
10. put three category/amount pairs (auto-sorted by key)
11. Print `map`
12. Cast to `TreeMap`; print `firstKey()`, `lastKey()`

STOP

4. UML Class Diagram

```
+-----+
| <<interface>> |
```



8. HashtableImplement.java — Hashtable

Source Code

```
package collectionspractice.classmethods;

import java.util.Hashtable;
import java.util.Map;

public class HashtableImplement {

    public static void show() {
        System.out.println("\n===### Hashtable Demo ###===");

        Map<String, Integer> gadgets = new Hashtable<>();
        //hashtable stores in two fields :keys and values
        //it does not allow duplicates
        gadgets.put("Laptop", 2);
        gadgets.put("Mobile", 10);
        gadgets.put("Tablet", 5);

        System.out.println("Stock table: " + gadgets);
    }
}
```

```

        System.out.println("Available laptops: " + gadgets.get("Laptop"));
        System.out.println("Contains key Mobile? " +
gadgets.containsKey("Mobile"));
        System.out.println("Contains value 5? " + gadgets.containsValue(5));

        // delete a mapping
        gadgets.remove("Tablet");
        System.out.println("After removing Tablet: " + gadgets);

        System.out.println("Total items: " + gadgets.size());
        System.out.println("Is map empty? " + gadgets.isEmpty());

        // get all keys and values
        System.out.println("Keys: " + gadgets.keySet());
        System.out.println("Values: " + gadgets.values());
    }
}

```

Hashtable is a **legacy, synchronized (thread-safe) Map implementation** — one of the original Java 1.0 classes, predating the modern Collection Framework. It stores key-value pairs like HashMap, but is thread-safe and, unlike HashMap, **does not allow null keys or null values**.

Method	What it does
<code>put(k, v)</code>	Inserts a key-value pair; throws NullPointerException if key or value is null
<code>get(k)</code>	Returns the value for a key
<code>containsKey()</code> / <code>containsValue()</code>	Checks presence of a key / value
<code>remove(k)</code>	Deletes a mapping by key
<code>size()</code> / <code>isEmpty()</code>	Element count / emptiness check
<code>keySet()</code> / <code>values()</code>	Views of keys / values

Exception: No exception occurs in this program — every key and value used ("Laptop", 2, etc.) is non-null. If the program had tried `gadgets.put(null, 5)` or `gadgets.put("Item", null)`, Hashtable would throw `NullPointerException` immediately (unlike `HashMap`, which allows one null key and multiple null values).

2. Step-wise Execution

1. `Hashtable<String,Integer> gadgets` created (via Map reference).
2. `put("Laptop",2), put("Mobile",10), put("Tablet",5)`
-> {Laptop=2, Mobile=10, Tablet=5} (order not guaranteed, and often different from `HashMap`'s iteration order)
3. Print stock table.
4. `get("Laptop") = 2`
5. `containsKey("Mobile") = true`

```

6. containsValue(5) = true
7. remove("Tablet") -> {Laptop=2, Mobile=10}
8. Print after removing Tablet.
9. size() = 2
10. isEmpty() = false
11. keySet() = [Laptop, Mobile] (order not guaranteed)
12. values() = [2, 10]

```

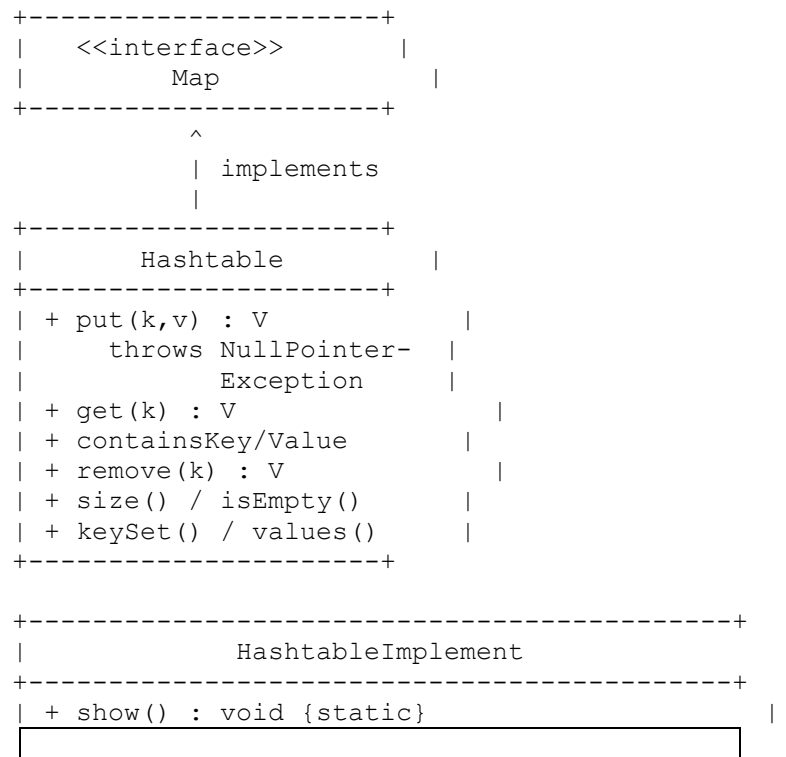
3. Algorithm

```

START
1. Create Hashtable gadgets (Map reference)
2. put three item/quantity pairs
3. Print map
4. Print get("Laptop"), containsKey("Mobile"), containsValue(5)
5. remove("Tablet"); print map
6. Print size(), isEmpty()
7. Print keySet(), values()
STOP

```

4. UML Class Diagram



✦ Result

The Java Collection Framework was successfully explored, organized by **class** rather than by interface, through a central driver class `CollectionFrameworkRunner` that invoked `show()` methods across seven demo classes — `ArrayListImplement`, `LinkedListImplement`, `VectorStackImplement`, `SetImplement`, `QueueDequeImplement`, `MapImplement`, and `HashtableImplement`. Each program correctly demonstrated the important methods of `ArrayList`, `LinkedList`, `Vector`, `Stack`, `HashSet`, `LinkedHashSet`, `TreeSet`, `PriorityQueue`, `ArrayDeque`, `HashMap`, `LinkedHashMap`, `TreeMap`, and `Hashtable` — confirming how each concrete class implements its interface with its own ordering behaviour (unordered, insertion-ordered, sorted, or heap-ordered) while sharing a common set of Collection Framework operations.